

Report

- **Introduction**

The “PingPong” lab consists of creating a reliable data network using the C programming language.

This report aims to describe the problem and how it was solved. It will also contain personal observations on the result obtained.

- **Objective**

The network allows data transmission between devices through the implementation of a server and a client. We implemented two types of communication: one using the UDP (User Datagram Protocol) protocol and one using the TCP (Transmission Control Protocol) protocol.

The differences between the two protocols are:

UDP: The UDP standard does not provide a stable and secure connection. Data is sent as independent packets, with no guarantee of delivery. The main characteristic of UDP is speed.

TCP: TCP connections are reliable because, before any data is sent, a stream connection is established between sender and receiver. TCP guarantees sequential delivery of the data.

- **Network implementation and difficulties encountered**

Among the first difficulties we encountered while carrying out the project was certainly understanding the code that had already been implemented.

As a first task, we developed a TCP client. While implementing the TCP socket, we came across some system calls we were not familiar with. We consulted the Linux manual to understand how they are used and how to implement them correctly. This process allowed us to become familiar with the new functions and to deepen our understanding of system calls in the context of programming.

The second step was creating a UDP client. Having become familiar with the system calls, it was easier to implement. However, the main difficulty was handling the timeout, which allows the client to retry sending packets if they are lost.

To complete the socket part, we had to implement the server so that a client-server connection could be simulated. The main problems occurred when accepting the connection: at first, because of an error in the bind, the accept system call was being interrupted. Another problem was the correct use of the getaddrinfo function, since the server's own address had to be assigned. This small misunderstanding caused the error in the bind and, consequently, in the accept. So once the first error was fixed, the following two were resolved as well.

- **Creating the graphs**

Finally, we were asked to implement a new bash script that would create a graph showing the logarithmic curve produced by the data generated by our code. We had problems from the implementation point of view, since we had never written in bash. Once we had gained some confidence with the language, we started working out the equations to compute the Bandwidth and Latency constants so as to produce the graph. The graph is obtained by computing the theoretical Throughput derived from the formula $f(x) = x / \text{latency} + x / \text{bandwidth}$; x indicates the message size.

- How we arrived at this formula

$$T(N) * D(N) = N$$

Throughput * delay = message size (bytes) → this formula was used to compute the delay for the maximum and the minimum message size, using the median throughput

$$D(N)_{max} = L + N_{max}/B$$

$$D(N)_{min} = L + N_{min}/B$$

Solving this linear system, we found the values of the bandwidth and the latency

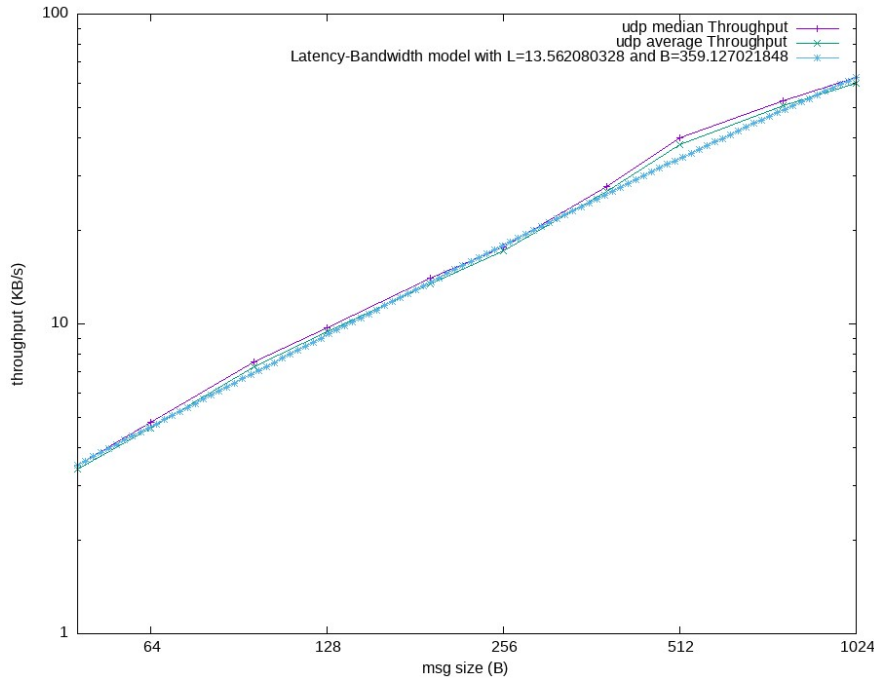
$$B = (N_{max} - N_{min}) / (D(N_{max}) - D(N_{min}))$$

$$L = (D(N_{min}) * N_{max} - D(N_{max}) * N_{min}) / (N_{max} - N_{min})$$

$$D(N) = N/T(D)$$

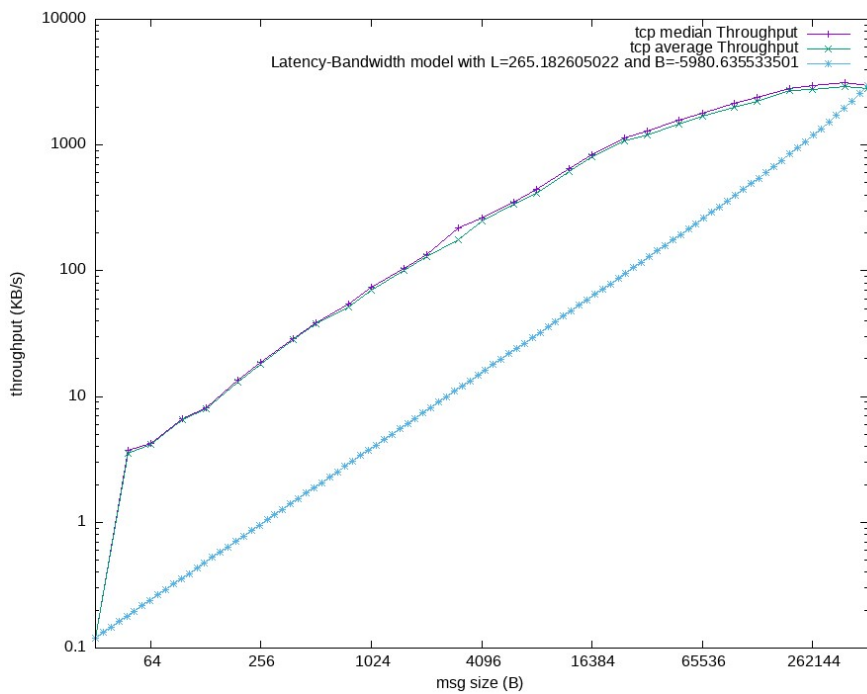
At this point it was easy to compute the function shown above by combining these two functions $D(N) = L + N/B$ and $T(N) * D(N) = N$

The graphs shown compare the median throughput, the average throughput and the bandwidth-latency model.



The UDP graph extends from 48 bytes to 1024 because larger datagrams were systematically blocked by the network

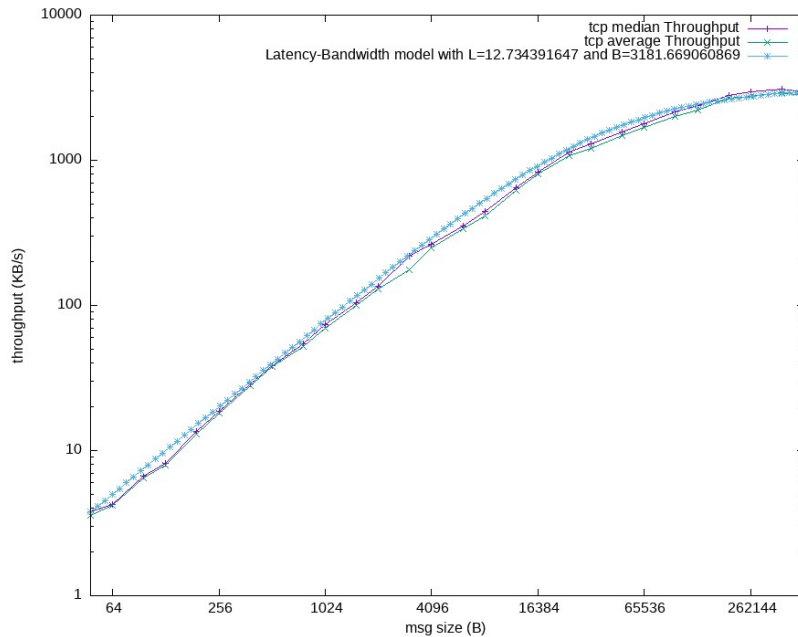
Note that at the start and end points of the graph the two lines intersect (the bandwidth-latency one and ours); naturally, at the intermediate values there are differences due both to our code and to the network



In this case we notice that the logarithmic curve of the bandwidth-latency graph looks more like a straight line. This is because the throughput at 32 is less than 1, producing a bandwidth less than zero, which is not allowed.

Also in this case, at the ends of the graph, the two lines intersect even though this is wrong

After several attempts to understand why the graphs did not even show a slight logarithmic curve, we decided to debug. The solution we found was to exclude communications smaller than 48 bytes, so as to make the bandwidth calculation more truthful. It is still unclear why this happens with both our `tcp_ping` executable and `gc_tcp_ping`.



*this version of the graph is created by
the script file `tcp_vers2.bash`*

Finally, after narrowing down the data to be considered, we obtained a more or less plausible result. Naturally, this graph also differs at the points in the middle of the lines, but they intersect at the ends.

- **Conclusion**

In conclusion, we believe that the lab was very useful and instructive, since it forced us to deal with languages of which we had only superficial knowledge.

The results of the graphs are, especially for UDP, not entirely reliable, but we decided to keep the partial experiment using `seti.dibris.unige.it` rather than run the experiment on localhost. This is because localhost made the results totally unreliable, since the bandwidth can reach enormous values and the latency will always be very close to 0.